

L12c: Deep Q-Learning

Q-networks and the DQN algorithm

CHEME 5800 Cornell University

Copyright Jeffrey Varner 2026. All Rights Reserved

Objectives for Today

How can an agent learn action values when its states are too many for a table?

Today's concepts

- Explain the limits of a Q-table: a table needs one entry for every state and action, and the number of states grows exponentially with the number of state variables.
- Train a Q-network with gradient descent: move each Q-value toward the reward plus the discounted value of the next state, and derive the gradient step for a linear network.
- Explain the replay buffer and the target network: the two parts that keep the training of a Q-network from becoming unstable.

Lecture: Deep Q-Learning

Lecture and Example

Lecture: Deep Q-Learning

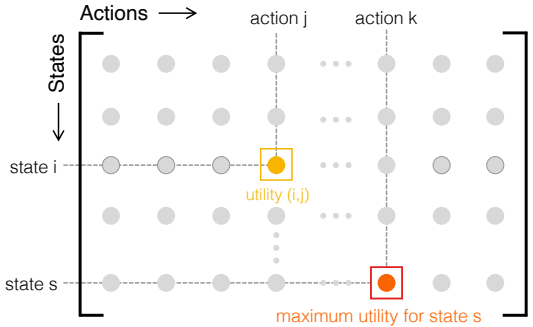
In this lecture, we replace the Q-table with a function that computes the action values from the state. This idea is the basis of deep Q-learning and of its best-known method, the deep Q-network (DQN). Today we start with the simplest such function, a linear one.

Example: Learn a dosing policy without a grid using a linear DQN

In this example, an agent learns to dose the patient of the dosing lab without rounding the tumor burden and the toxicity to grid levels. Its linear Q-network has far fewer weights than the dosing lab's Q-table has entries.

The Q-Table

Tabular Q-learning stores one action value for every state-action pair.



Entry (i, j) holds the value (utility) of taking action j in state i. The policy reads each row and picks the column with the largest entry.

The Curse of Dimensionality

The Q-table has three problems:

- The table grows exponentially: a state with m variables, each taking L values, has L^m states. The grid world has $20^2 = 400$ states. Adding a battery level with 20 values gives $20^3 = 8000$.
- The table does not generalize: each entry learns only from visits to its own state and action. The roomba must visit every square to learn its value.
- Rounding changes the problem: rounding treats different states as the same state. The dosing lab's policy gives no dose at a tumor burden of 0.109, because its grid rounds it to 0.100.

Richard Bellman called this exponential growth the curse of dimensionality.

Q-Networks

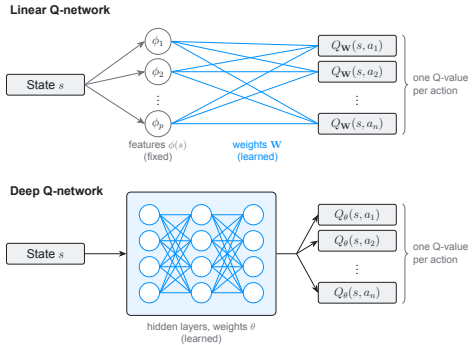
Instead, we replace the table with a function $Q_\theta : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ with parameters θ. Each state is a vector of m state variables, $s \in \mathcal{S} \subseteq \mathbb{R}^m$, and we do not need to enumerate the states.

A Q-network takes a state s and returns one value for each of the n actions in $\mathcal{A} = \{a_1, \ldots, a_n\}$:

$$(Q_\theta(s, a_1), \ldots, Q_\theta(s, a_n)) \in \mathbb{R}^n, \qquad \pi(s) = \arg\max_{a \in \mathcal{A}} Q_\theta(s, a)$$

The number of weights depends on the features, layers, and actions we choose, not on the number of states.

Two Q-Networks



In both networks, training estimates only the weights. We choose everything else:

- Linear: we choose the p features $\phi_1(s), \ldots, \phi_p(s)$.
- Deep: we choose the layers, the units per layer, and their functions. Training estimates the weights of every layer.

A Linear Q-Network

A linear Q-network first maps the state to a feature vector $\phi(s) = \big(\phi_1(s), \dots, \phi_p(s)\big) \in \mathbb{R}^p$. Each feature $\phi_j : \mathcal{S} \to \mathbb{R}$ is a fixed function of the state that we choose. The Q-value of action a is a weighted sum of the features:

$$Q_{\mathbf{W}}(s, a) = \mathbf{w}_a^\top \phi(s) = \sum_{j=1}^{p} w_{a,j}\, \phi_j(s)$$

where $w_{a,j}$ is the weight of action a on feature $\phi_j(s)$. The weight matrix $\mathbf{W}$ has one row $\mathbf{w}_a^\top$ for each of the n actions, which gives $n \cdot p$ weights in all.

Two Choices of Features

Radial basis functions with centers $\mathbf{c}_j$ and width σ:

$$\phi_j(s) = \exp\left(-\frac{\|s - \mathbf{c}_j\|_2^2}{2\sigma^2}\right)$$

We choose the centers, often on an even grid, and σ close to the grid spacing. A weight update at one state also changes the Q-values of nearby states. An even grid needs L^m centers, which brings back the curse of dimensionality.

One-hot: number the K states, and let $\phi(s_k) = \mathbf{e}_k$, a one in entry k and zeros elsewhere:

$$Q_{\mathbf{W}}(s_k, a) = \mathbf{w}_a^\top \mathbf{e}_k = w_{a,k}$$

Each weight is one table entry, $w_{a,k} = Q(s_k, a)$. Thus, $\mathbf{W} = \mathbf{Q}^\top$, and a Q-table is a linear network.

A Deep Q-Network

A feedforward neural network has an input layer, one or more hidden layers, and an output layer, with no cycles:

- Input layer: holds the state variables $s \in \mathbb{R}^m$.
- Hidden layers: each node computes a weighted sum of the layer before it, plus a bias, then applies an activation function such as ReLU$(x) = \max(0, x)$.
- Output layer: each of the n nodes computes a weighted sum, with no activation function. A Q-value can take any sign.

The parameters θ are the weights and biases of every layer. Mnih et al. (2015) trained a deep Q-network to play Atari games from the pixels on the screen.

The DQN Loss

The training data are transitions $(s_i, a_i, r_i, s'_i, d_i)$: a state, the action taken, the reward, the next state, and a done flag d_i.

- The flag is one if s'_i is absorbing, a state that ends the episode, such as a cure or a toxic death, and zero otherwise.
- An episode that stops at a step limit keeps $d_i = 0$ on its last transition.

Training uses gradient descent. One step with learning rate α is $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$, where $\nabla_\theta \mathcal{L}$ is the vector of derivatives of the loss with respect to the parameters.

DQN Bellman Loss

The target y_i of transition i is the value that the prediction $Q_\theta(s_i, a_i)$ should match:

$$y_i = r_i + \gamma\,(1 - d_i) \max_{a' \in \mathcal{A}} Q_{\theta^-}(s_i', a')$$

where θ^- are the parameters of the target network, a delayed copy of the Q-network, and γ is the discount factor.

On a mini-batch of N_{batch} past transitions, the loss is:

$$\mathcal{L}(\theta) = \frac{1}{2N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} \Big(\underbrace{y_i - Q_\theta(s_i, a_i)}_{\text{prediction error}} \Big)^2$$

The prediction uses the action a_i that was actually taken.

The Gradient of a Linear Network

Two facts make the gradient simple. The prediction $Q_{\mathbf{W}}(s_i, a_i) = \mathbf{w}_{a_i}^\top \phi(s_i)$ is linear in the weights. The target y_i uses the target network's weights $\mathbf{W}^-$, so it does not depend on $\mathbf{W}$.

$$\nabla_{\mathbf{w}_a} \mathcal{L} = \frac{1}{2N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} \nabla_{\mathbf{w}_a} \left(y_i - \mathbf{w}_{a_i}^\top \phi(s_i) \right)^2 \qquad \text{gradient of each term}$$

$$= \frac{1}{2N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} 2 \left(y_i - \mathbf{w}_{a_i}^\top \phi(s_i) \right) \left(-\nabla_{\mathbf{w}_a} \mathbf{w}_{a_i}^\top \phi(s_i) \right) \qquad \text{chain rule}$$

$$= -\frac{1}{N_{\text{batch}}} \sum_{i \,:\, a_i = a} \left(y_i - \mathbf{w}_a^\top \phi(s_i) \right) \phi(s_i) \qquad \text{only } a_i = a \text{ counts}$$

The last sum runs only over the transitions whose action a_i equals a. If a mini-batch has the actions (a_2, a_1, a_2, a_3), the gradient for $\mathbf{w}_{a_2}$ adds up transitions 1 and 3 only.

The Gradient Step of a Linear Network

One gradient step, $\mathbf{w}_a \leftarrow \mathbf{w}_a - \alpha \nabla_{\mathbf{w}_a} \mathcal{L}$, updates the row of each action in the mini-batch:

$$\mathbf{w}_a \leftarrow \mathbf{w}_a + \frac{\alpha}{N_{\text{batch}}} \sum_{i \,:\, a_i = a} \left(y_i - \mathbf{w}_a^\top \phi(s_i)\right) \phi(s_i)$$

- First, compute the prediction error of every transition in the mini-batch with the current weights.
- Then, update each row $\mathbf{w}_a$ once, using the errors of the transitions that took action a.

Stabilizer 1: The Replay Buffer

Tabular Q-learning converges if it visits every state-action pair often enough. Deep Q-learning can diverge, with Q-values that grow without bound (Sutton and Barto, 2018). Two additions make divergence much less likely.

The replay buffer $\mathcal{D}$ is the agent's memory of its experience. It stores the most recent M transitions, the training data:

- Each training step draws a random mini-batch of N_{batch} transitions. This mixes old and new experience from many episodes.
- Training waits for N_{warm} transitions, the warm-up, so the first mini-batches are not a few similar transitions.

Stabilizer 2: The Target Network

The target network $Q_{\theta^-}(s, a)$ is a frozen copy of the Q-network. It computes the targets y_i, and training does not change its weights θ^-.

- How we use it: every C steps, counted over all episodes, the algorithm copies the Q-network into the target network, $\theta^- \leftarrow \theta$. Between copies, the targets stay fixed.
- Why we need it: without it, the Q-network would compute its own targets. Each gradient step would then move the targets as well as the predictions.

DQN Algorithm: Initialize

Initialize: Given a Q-network $Q_\theta(s, a)$, set θ to zeros for a linear network or to small random values for a deep network, and copy the parameters to the target network, $\theta^- \leftarrow \theta$. Start with an empty replay buffer $\mathcal{D}$ and a total step counter $\tau \leftarrow 0$.

Hyperparameters:

- the discount factor $0 \leq \gamma < 1$ and the learning rate $\alpha > 0$,
- the exploration probability ϵ of each episode,
- the number of episodes N and the maximum number of steps per episode $t_{\max}$,
- the buffer size M and the warm-up size N_{warm}, the number of transitions the buffer must hold before training begins,
- the mini-batch size N_{batch} and the target-network copy interval C.

DQN Algorithm

For each of the N episodes **do**:

1. Choose a starting state s that is not absorbing, and set the step counter $t \leftarrow 0$.
2. While s is not absorbing and $t < t_{\max}$ **do**:
   - 2.1 Choose the action a: draw a random number ρ uniformly from $[0, 1]$.
     - If $\rho \leq \epsilon$ **then** choose $a \in \mathcal{A}$ uniformly at random (exploration).
     - Otherwise, choose the greedy action $a \leftarrow \arg\max_{a' \in \mathcal{A}} Q_\theta(s, a')$ (exploitation).
   - 2.2 Take action a, and observe the reward r, the next state s', and the done flag d from the *world*, where $d = 1$ if s' is absorbing and $d = 0$ otherwise.
   - 2.3 Store the transition (s, a, r, s', d) in the replay buffer $\mathcal{D}$, and remove the oldest transition if $\mathcal{D}$ holds more than M.
   - 2.4 Update the counters, $t \leftarrow t + 1$ and $\tau \leftarrow \tau + 1$.
   - 2.5 If $\mathcal{D}$ holds at least N_{warm} transitions **then** take one training step.
   - 2.6 If τ is a multiple of C **then** copy the Q-network to the target network, $\theta^- \leftarrow \theta$.
   - 2.7 Update the state, $s \leftarrow s'$.

Extract Policy: For any state s, compute $\pi(s) \leftarrow \arg\max_{a \in \mathcal{A}} Q_\theta(s, a)$.

DQN Algorithm: The Training Step

Step 2.5 of the loop takes one training step:

1. Draw a mini-batch of N_{batch} transitions $(s_i, a_i, r_i, s'_i, d_i)$, $i = 1, \ldots, N_{\text{batch}}$, uniformly at random with replacement from $\mathcal{D}$.
2. Compute the target of each transition with the *target network*, $y_i \leftarrow r_i + \gamma\,(1 - d_i)\,\max_{a' \in \mathcal{A}} Q_{\theta^-}(s'_i, a')$.
3. Compute the loss on the actions that were actually taken:

$$\mathcal{L}(\theta) = \frac{1}{2N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} \left(y_i - Q_\theta(s_i, a_i)\right)^2$$

4. Take a *single* gradient descent step on the loss, $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}(\theta)$.
   - *Why only a single step?* Many steps on one small mini-batch can overfit its few transitions and undo what the network learned from the rest of the buffer.

Notes on the Algorithm

- The policy gives an action for any state, including states the agent never visited.
- *Why random starting weights for a deep network?* If all the weights start equal, the units of a hidden layer compute the same feature, get the same update, and stay equal. A linear network has fixed features, and zeros work.
- In the example, a linear network with radial basis features trains quickly on a laptop. The same algorithm trains a deep network, which needs a package for neural networks and much more time.

Example: A Linear DQN for Adaptive Dosing

In this example, an agent learns to dose the patient of the dosing lab without a grid:

1. Build the patient model, with the tumor burden and the toxicity kept as continuous numbers.
2. Train a linear DQN with radial basis features.
3. Compare its doses with value iteration on the dosing lab's 41-level grid and on a 201-level grid.

From the dosing lab's starting state, the linear DQN cures the patient in 23 cycles and the 201-level policy in 19. The 41-level policy does not cure the patient within 100 cycles.

Example: A linear DQN for adaptive drug dosing

Summary

We replaced the Q-table with a Q-network and built deep Q-learning around it.

Key takeaways

- **Q-networks.** A Q-network computes every action value from features of the state, with a number of parameters that does not grow with the number of states.
- **Gradient descent.** Each step moves the predictions toward the targets. For a linear network, it adds each prediction error times the features to the weights of the action taken.
- **Stabilizers.** Random mini-batches from the replay buffer mix many episodes, and the target network keeps the targets fixed between copies.

A linear network with well-chosen features is often enough for a problem with a few state variables. In a deep network, the features come from hidden-layer weights estimated from data.